

实验 2：组合逻辑部件设计实验报告

姓名： 学号：

2026 年 4 月 13 日

1 实验目的

1. 掌握译码器、编码器、数码管等组合逻辑部件的设计方法。
2. 掌握 1 位全加器与 4 位串行进位加法器的设计方法。
3. 掌握补码减法运算的实现方法，理解加减法统一实现思想。
4. 掌握汉明码编码、检错与纠错电路的设计思路。
5. 掌握桶形移位器的结构设计与工作原理。

2 实验环境

- 软件环境：Logisim 2.16
- 实验平台：数字逻辑与计算机基础实验平台

3 实验内容与结果分析

3.1 3-8 译码器实验

3.1.1 实验原理

本实验要求根据 74X138 的工作原理，设计一个由独立逻辑门构成的 3-8 译码器。译码器共有 3 个地址输入端 A, B, C ，3 个使能控制端 $G1, G2A_L, G2B_L$ ，以及 8 个输出端 $Y0_L \sim Y7_L$ 。

当且仅当

$$G1 = 1, \quad G2A_L = 0, \quad G2B_L = 0$$

时，译码器处于使能状态。此时输入的三位二进制数 CBA 决定哪一路输出有效。由于 74X138 为低电平有效器件，因此仅有一路输出为低电平，其余输出均为高电平。

例如：

$$CBA = 000 \Rightarrow Y0_L = 0$$

$$CBA = 001 \Rightarrow Y1_L = 0$$

...

$$CBA = 111 \Rightarrow Y7_L = 0$$

3.1.2 电路设计

本实验利用与非门、非门和与门实现 3-8 译码器逻辑。通过对输入地址位及其反相信号进行组合，生成八路低电平有效输出。

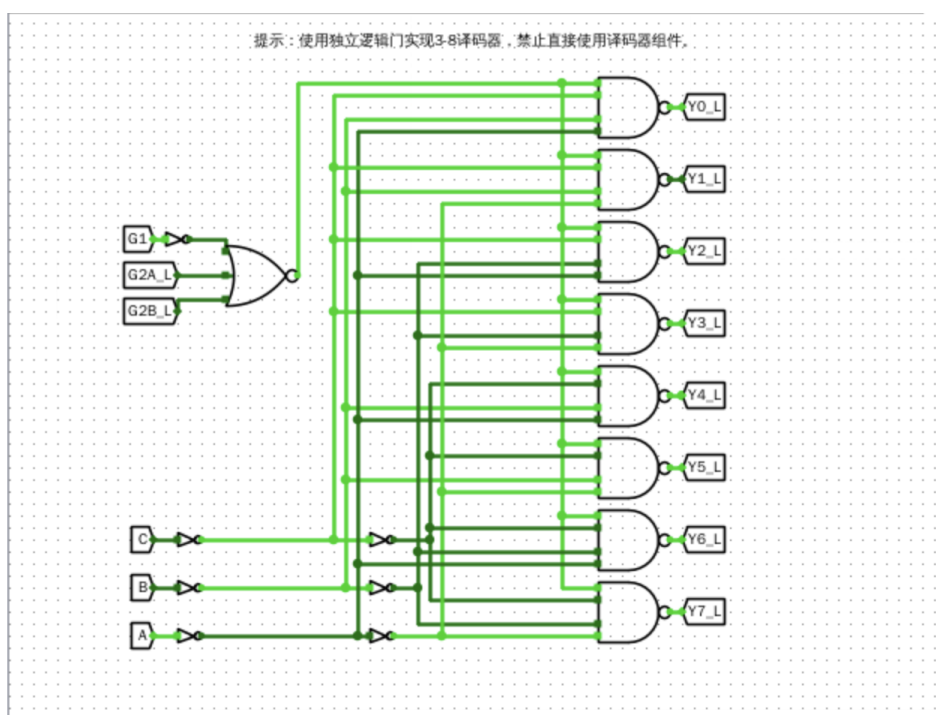


图 1: 3-8 译码器电路图

3.1.3 仿真测试与分析

在 Logisim 中改变输入信号 $G1, G2A_L, G2B_L, A, B, C$ 的取值，观察各输出端状态。仿真结果表明，当使能条件满足时，输出端中仅有一路为低电平，且对应输入地址的十进制值；当使能条件不满足时，各输出均为高电平，电路行为与理论一致。

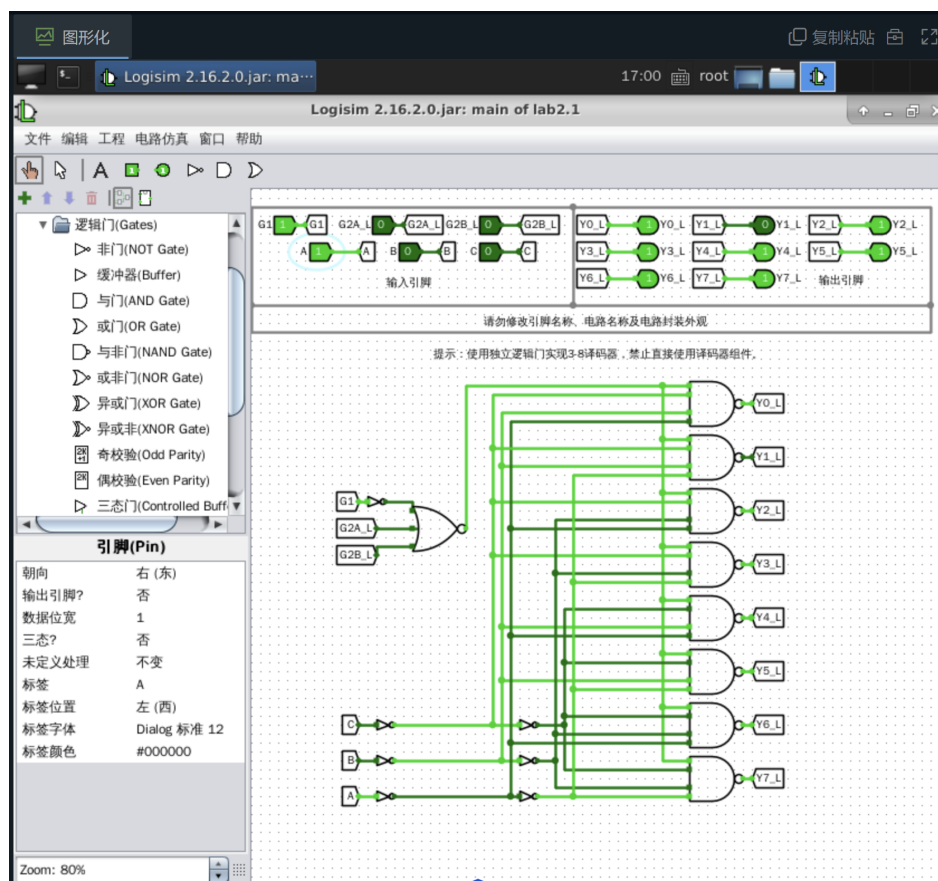


图 2: 3-8 译码器仿真图

3.2 8-3 优先级编码器实验

3.2.1 实验原理

8-3 优先级编码器的功能是将 8 路输入信号编码为 3 位二进制输出。其特点在于：当多个输入同时为有效时，输出优先级最高的那一路编号。

设输入端为 $I_0 \sim I_7$ ，输出端为 O_0, O_1, O_2 ，则编码器满足：

- 若仅 $I_0 = 1$ ，则输出为 000；
- 若仅 $I_1 = 1$ ，则输出为 001；
- ...
- 若仅 $I_7 = 1$ ，则输出为 111。

若多个输入同时有效，则由优先级最高的输入决定输出编码。本实验还将输出结果连接到十六进制数码管，以直观显示编码结果。

3.2.2 电路设计

本实验使用与门、或门、输入引脚、分线器和十六进制数码显示器完成电路搭建。各输入端经过优先级逻辑组合后产生 3 位编码输出。

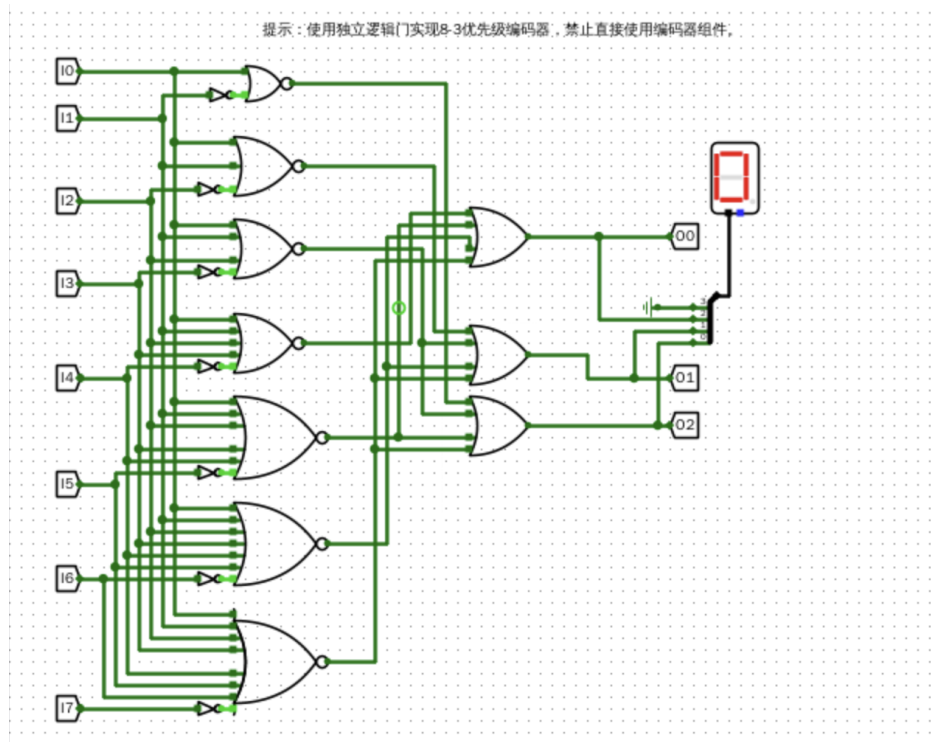


图 3: 8-3 优先级编码器电路图

3.2.3 仿真测试与分析

通过逐步改变输入端状态，观察输出编码及数码管显示，可验证编码器功能正确。当有多路输入同时有效时，输出结果符合“优先级最高输入获胜”的设计要求。

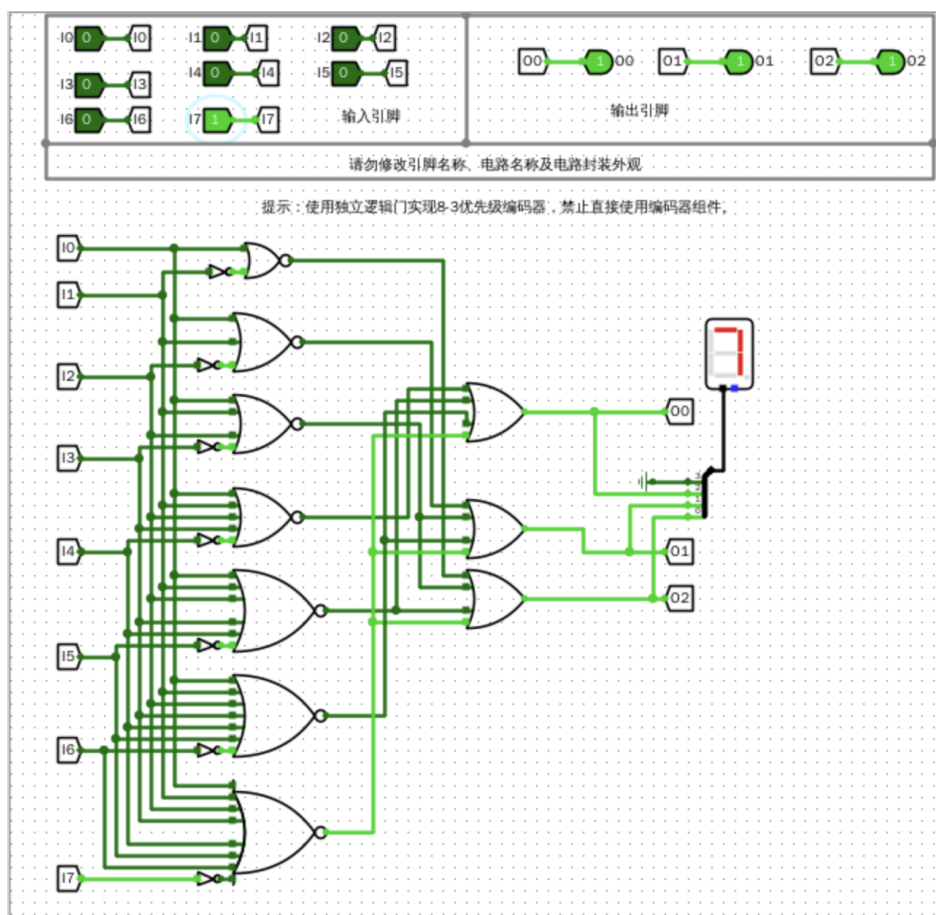


图 4: 8-3 优先级编码器仿真图

3.3 全加器与 4 位串行加减法器实验

3.3.1 1 位全加器原理

全加器有三个输入端 A, B, Cin ，两个输出端 $F, Cout$ 。其中， F 为本位和， $Cout$ 为向高位产生的进位。逻辑表达式为：

$$F = A \oplus B \oplus Cin$$

$$Cout = AB + ACin + BCin$$

根据该表达式，可以用异或门、与门、或门搭建全加器。全加器真值表如表 1 所示。

表 1: 全加器真值表

A	B	C_{in}	F	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

3.3.2 全加器电路实现

按照上述逻辑表达式，先在子电路中设计 1 位全加器，为后续构造 4 位串行进位加法器做准备。

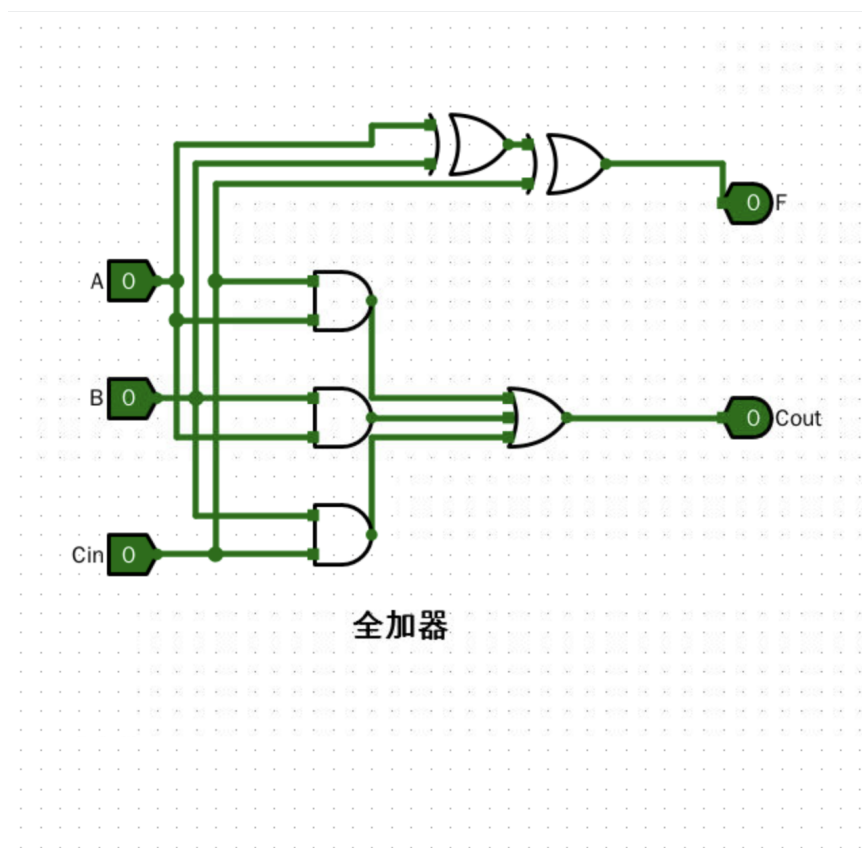


图 5: 1 位全加器电路图

3.3.3 4 位串行加减法器原理

通过串联 4 个 1 位全加器子电路，可以得到 4 位串行进位加法器。若进一步在被加数输入侧加入异或控制，并将控制信号接入最低位进位输入 Cin ，则可将加法与减法统一起来实现：

$$Cin = 0 \Rightarrow F = X + Y$$

$$Cin = 1 \Rightarrow F = X + \bar{Y} + 1 = X - Y$$

因此，当控制位为 0 时执行补码加法，当控制位为 1 时执行补码减法。

3.3.4 4 位加减法器电路

本实验在 4 个全加器级联基础上实现了 4 位加减法器，并将输入与输出连接到数码显示器进行验证。

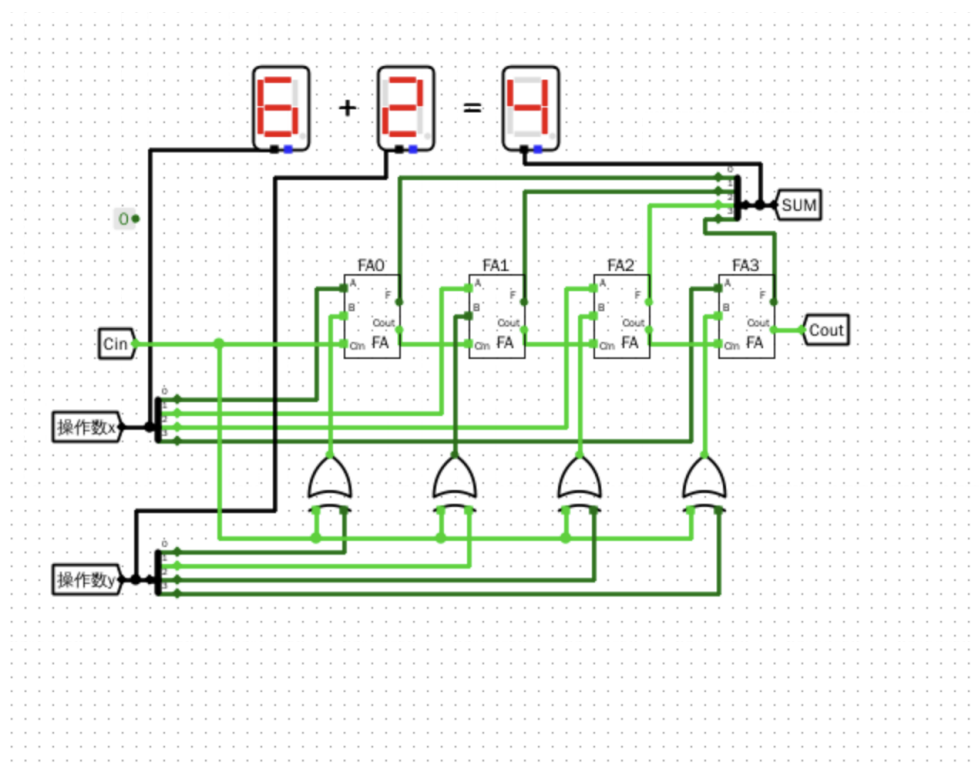


图 6: 4 位串行加减法器电路图

3.3.5 仿真测试与分析

对不同输入数据和控制信号进行测试，结果表明该电路既能实现普通加法，也能实现补码减法。输出结果与理论计算一致，说明设计正确。

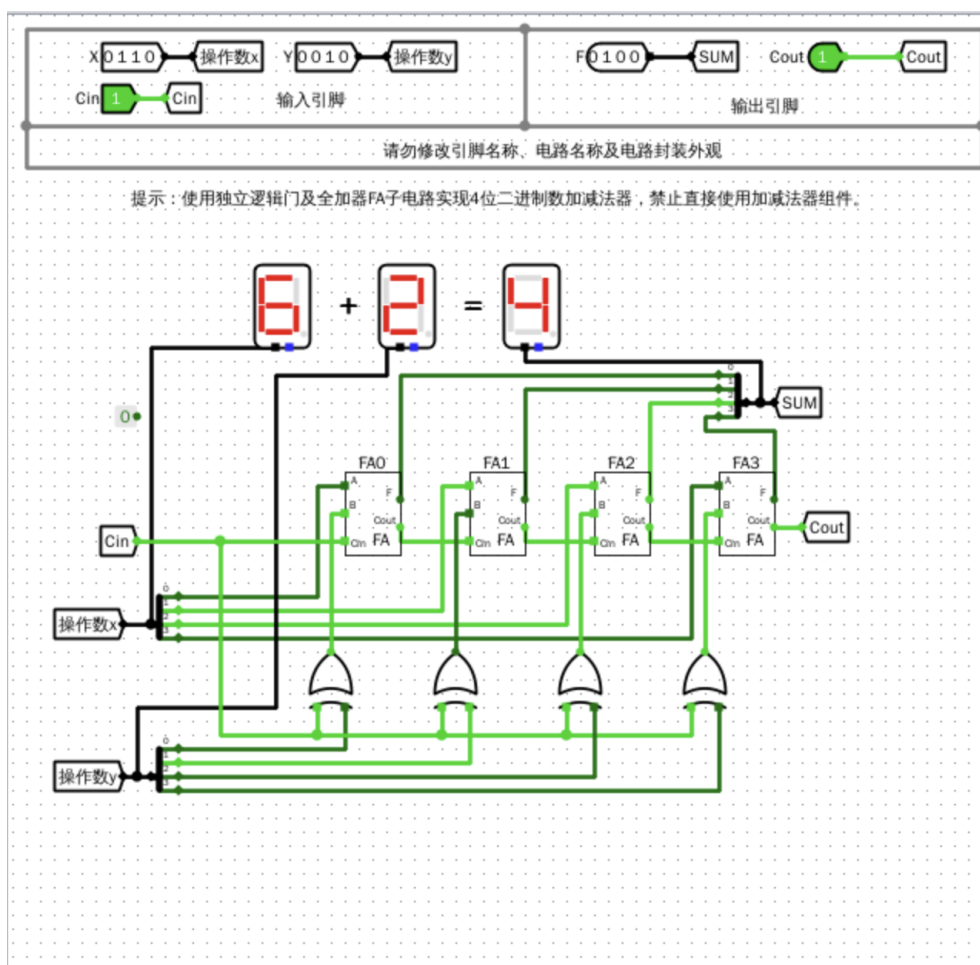


图 7: 4 位串行加减法器仿真图

3.4 7 位汉明码检/纠错电路实验

3.4.1 实验原理

汉明码是一种能够实现单比特纠错的编码方式。对于 4 位数据位 M_4, M_3, M_2, M_1 ，可设置 3 位校验位 P_3, P_2, P_1 ，形成 7 位码字。按实验要求，码字位置安排为：

$$P_1 P_2 M_1 P_3 M_2 M_3 M_4$$

根据偶校验规则，有：

$$P_1 = M_1 \oplus M_2 \oplus M_4$$

$$P_2 = M_1 \oplus M_3 \oplus M_4$$

$$P_3 = M_2 \oplus M_3 \oplus M_4$$

当接收端得到新的数据位与校验位后，可构造故障字：

$$S_1 = M_1 \oplus M_2 \oplus M_4 \oplus P_1$$

$$S2 = M1 \oplus M3 \oplus M4 \oplus P2$$

$$S3 = M2 \oplus M3 \oplus M4 \oplus P3$$

故障字 $S3S2S1$ 的数值即可指示出错位置；若故障字为 000，则表示无错。

3.4.2 4 位偶校验子电路

在整个汉明码检错与纠错电路中，需要先封装 4 位偶校验器子电路，以便分别计算三组校验关系。

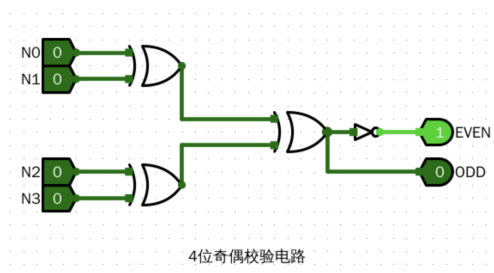


图 8: 4 位偶校验子电路

3.4.3 3-8 译码子电路

汉明码故障字经过译码后可定位到具体出错的位，因此还需要封装 3-8 译码器子电路供纠错部分使用。

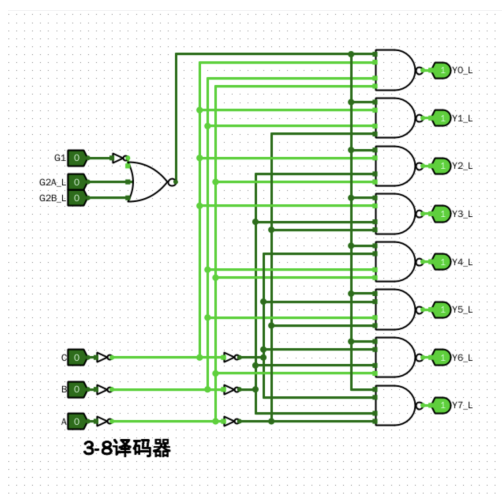


图 9: 汉明码实验中使用的 3-8 译码子电路

3.4.4 汉明码检/纠错电路实现

将偶校验子电路、3-8 译码器、异或门、分线器与显示元件组合，即可完成 7 位汉明码检/纠错电路。由于 3-8 译码器输出为低电平有效，因此在接入后续异或修正逻辑前需要适当取反处理。

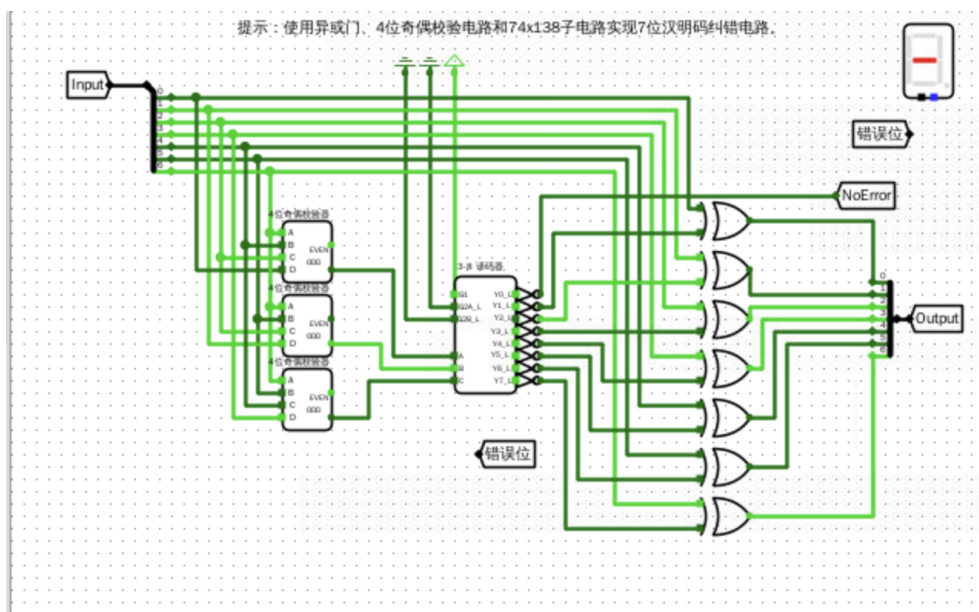


图 10: 7 位汉明码检/纠错电路图

3.4.5 仿真测试与分析

输入若干测试码字进行验证，电路能够正确给出是否出错以及出错位置，并对单比特错误进行纠正，说明设计满足实验要求。

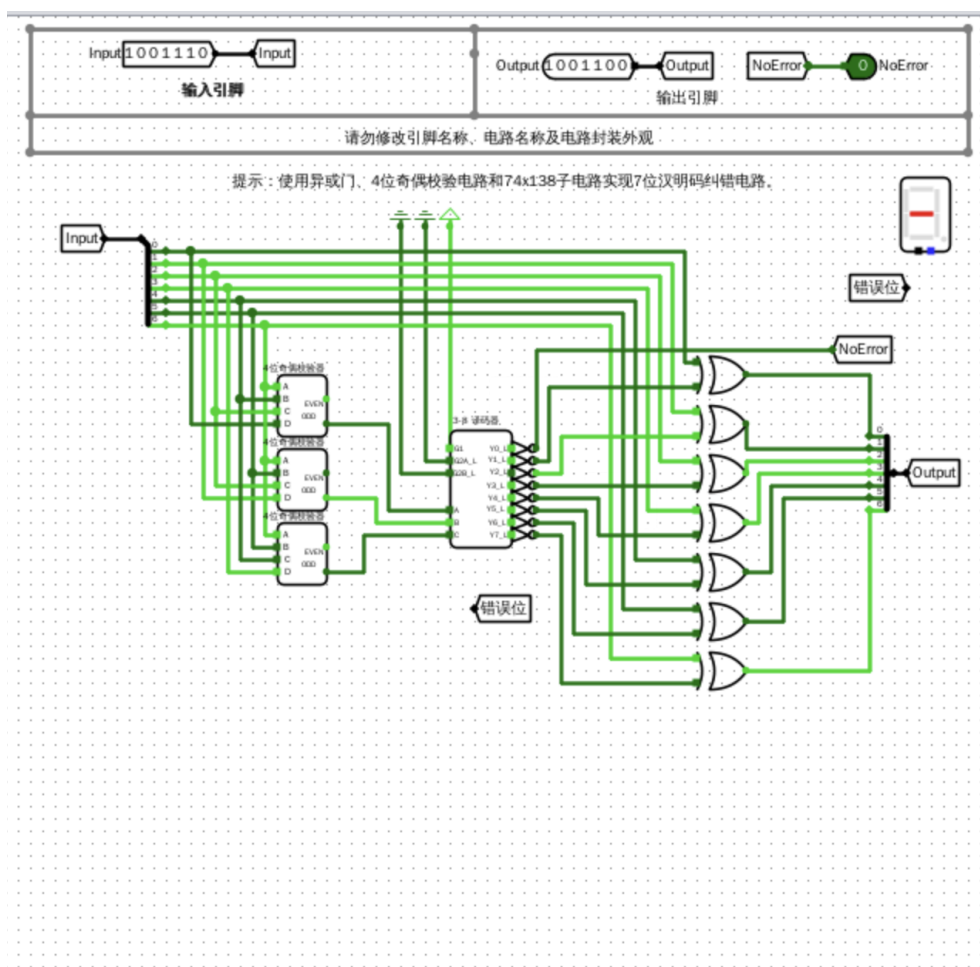


图 11: 7 位汉明码检/纠错仿真图

3.5 8 位桶形移位器实验

3.5.1 实验原理

桶形移位器是一种组合逻辑移位器，能够在—个时钟周期内完成多位移位操作。实验中的 8 位桶形移位器输入端为 $din[7:0]$ ，输出端为 $dout[7:0]$ ，位移量为 $shamt[2:0]$ ，并有两个控制信号：

- L/R ：控制左移或右移；
- A/L ：控制算术/循环移位或逻辑移位。

根据实验分析，8 位桶形移位器可采用三级结构实现：

- 第一级由 $shamt[0]$ 控制是否移动 1 位；
- 第二级由 $shamt[1]$ 控制是否移动 2 位；
- 第三级由 $shamt[2]$ 控制是否移动 4 位。

右移时：

- $A/L = 1$ 表示算术右移，移入符号位；
- $A/L = 0$ 表示逻辑右移，移入 0。

左移时：

- $A/L = 1$ 表示循环左移；
- $A/L = 0$ 表示逻辑左移，移入 0。

3.5.2 电路设计

本实验使用多路选择器、分线器和常量 0 等元件构成三级桶形移位结构。

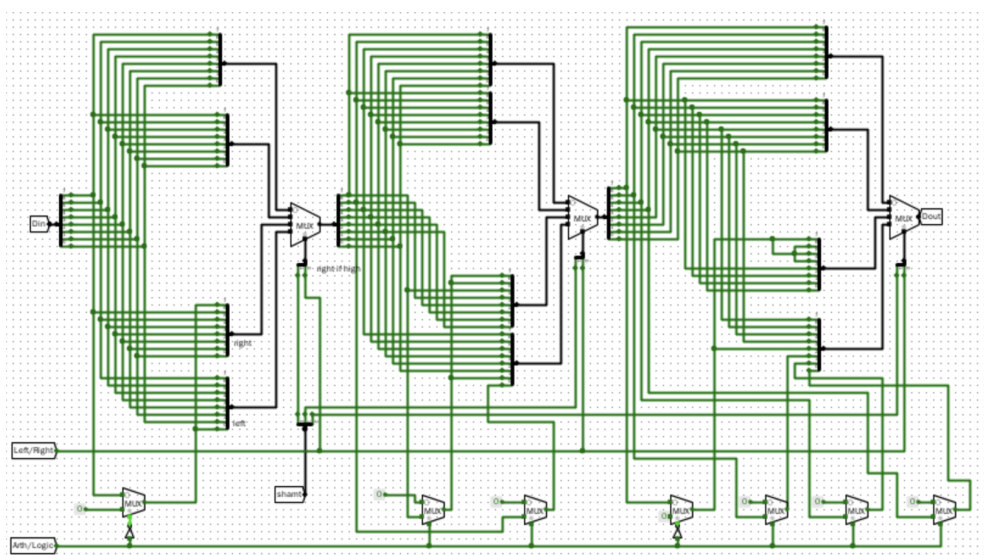


图 12: 8 位桶形移位器电路图

3.5.3 仿真测试与分析

通过输入不同的 8 位数据与控制信号，测试逻辑右移、算术右移、逻辑左移与循环左移等功能。实验结果表明，桶形移位器能够根据控制端正确完成不同方向与类型的移位操作。

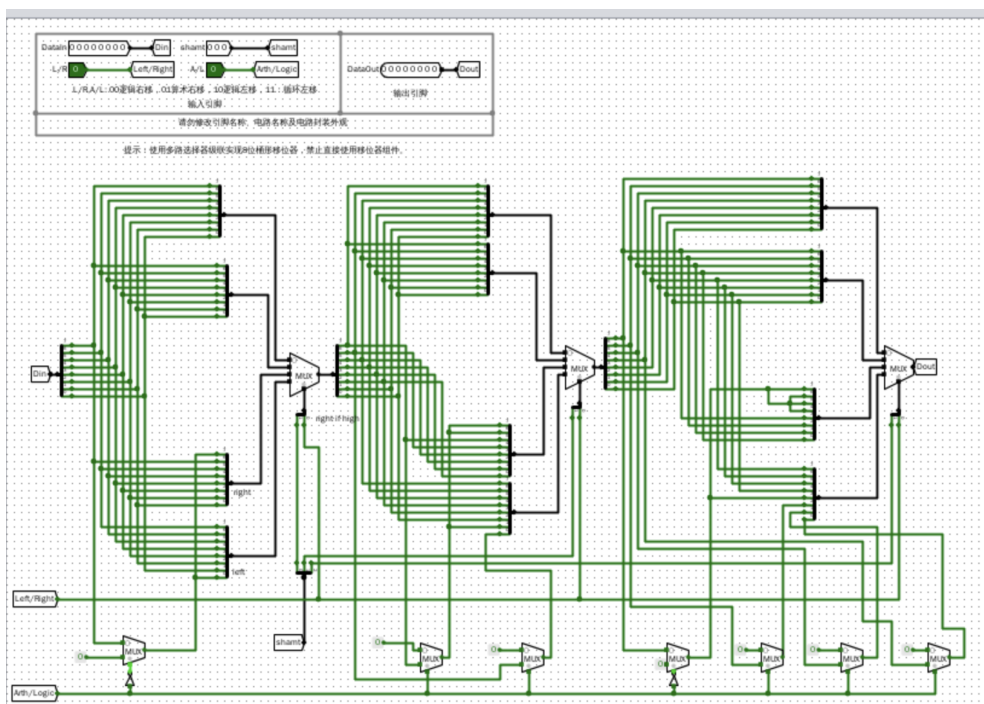


图 13: 8 位桶形移位器仿真图

4 实验中的错误现象及分析

在实验过程中，主要遇到了以下几类问题：

4.1 连线错误导致功能异常

在较复杂的电路中，如汉明码检/纠错电路和桶形移位器，连线较多，若某一输入端或分线器接错，会导致整体功能异常。

原因分析：

- 输入位序与逻辑表达式中的变量顺序不一致；
- 子电路封装后引脚方向或顺序设置错误；
- 分线器位宽或方向设置不正确。

解决方法：

- 为所有输入输出端添加明确标识符；
- 使用探针逐位检查信号；
- 子电路封装后重新核对引脚定义。

4.2 低电平有效信号理解错误

在 3-8 译码器与汉明码电路中，译码器输出为低电平有效，若直接按高电平有效理解，容易导致后级逻辑结果错误。

原因分析：未充分注意实验指导中对 74X138 输出有效电平的说明。

解决方法：在接入后级逻辑前，根据实际需要增加反相处理，并在分析真值表时明确“有效”对应的是低电平。

4.3 加减法控制位理解不清

在 4 位加减法器中，若未正确理解控制位与补码减法的关系，容易将减法实现错误。

原因分析：补码减法本质上是“按位取反加一”，需要同时控制输入异或与最低位进位输入。

解决方法：将控制信号同时连接到被减数修正逻辑与最低位 C_{in} ，从而保证减法操作正确实现。

5 实验总结

通过本次实验，我完成了译码器、优先级编码器、全加器及加减法器、汉明码检纠错电路、桶形移位器等多种组合逻辑部件的设计与验证。实验过程中，我进一步掌握了以下内容：

1. 根据功能要求推导逻辑关系并设计组合逻辑电路；
2. 利用子电路封装和模块复用构建更复杂系统；
3. 理解低电平有效器件与普通逻辑门电路之间的联系；
4. 理解补码加减法统一实现的原理；
5. 初步掌握汉明码纠错与桶形移位器的结构设计思想。

本次实验将理论分析与具体电路实现相结合，使我对组合逻辑系统的设计方法有了更全面的认识。

6 思考题

6.1 1. 修改实验中的加法器电路，生成进位标志 CF、溢出标志 OF、符号标志 SF 和结果为零标志位 ZF

设 4 位加减法器输出结果为 $F[3:0]$ ，最高位为 F_3 ，最高位进位输入与进位输出分别为 C_3 和 C_4 ，则各标志位可定义如下：

(1) **进位标志 CF** 对于无符号运算，进位标志等于最高位向外输出的进位：

$$CF = C_4$$

(2) **溢出标志 OF** 对于补码有符号数运算，溢出标志由最高位输入进位和输出进位异或得到：

$$OF = C_3 \oplus C_4$$

(3) **符号标志 SF** 符号标志等于结果最高位：

$$SF = F_3$$

(4) **零标志 ZF** 当结果 4 位全为 0 时，零标志为 1：

$$ZF = \overline{F_3 + F_2 + F_1 + F_0}$$

实现时可在原加法器基础上引出最高位进位，并增加一个或门加非门电路检测零结果。

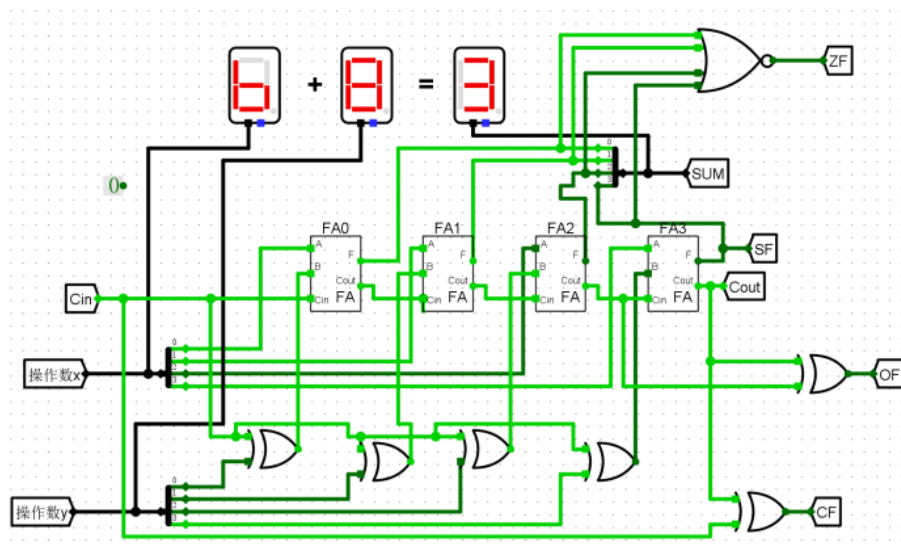


图 14: 思考题 1

6.2 2. 通过带标志位加法器输出，说明如何判断 $X > Y$ 和 $X < Y$

比较运算通常通过减法实现，即计算：

$$X - Y$$

(1) 无符号数比较 无符号数比较主要看 CF 与 ZF ：

- 若 $ZF = 1$ ，则 $X = Y$ ；
- 若 $ZF = 0$ 且 $CF = 0$ ，则 $X > Y$ ；
- 若 $ZF = 0$ 且 $CF = 1$ ，则 $X < Y$ 。

(2) 有符号数比较 有符号数比较主要看 SF 、 OF 与 ZF ：

- 若 $ZF = 1$ ，则 $X = Y$ ；
- 若 $ZF = 0$ 且 $SF \oplus OF = 0$ ，则 $X > Y$ ；
- 若 $ZF = 0$ 且 $SF \oplus OF = 1$ ，则 $X < Y$ 。

因此，在同一套减法器输出基础上，通过不同标志位组合即可完成无符号与有符号比较，实现电路与思考题 1 一致

6.3 3. 不使用加法器，直接使用逻辑门实现 4 位无符号二进制数比较器

设两个 4 位无符号数分别为：

$$X = X_3X_2X_1X_0, \quad Y = Y_3Y_2Y_1Y_0$$

比较大小时应从高位到低位依次判断。定义每一位相等信号：

$$E_i = X_iY_i + \overline{X_i}\overline{Y_i}$$

则“大于”输出可写为：

$$G = X_3\overline{Y_3} + E_3X_2\overline{Y_2} + E_3E_2X_1\overline{Y_1} + E_3E_2E_1X_0\overline{Y_0}$$

“小于”输出可写为：

$$L = \overline{X_3}Y_3 + E_3\overline{X_2}Y_2 + E_3E_2\overline{X_1}Y_1 + E_3E_2E_1\overline{X_0}Y_0$$

据此可直接使用与门、或门、非门构成一个 4 位无符号比较器，不必借助加法器或减法器。

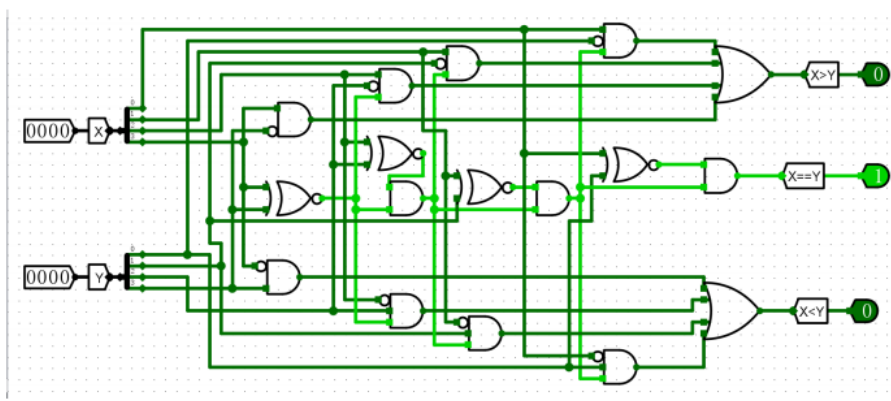


图 15: 思考题 3

6.4 4. 如何使用 8 位桶形移位器扩展到 32 位桶形移位器

将 8 位桶形移位器扩展到 32 位桶形移位器，核心思想是保持“分级移位”的结构不变，只扩大数据宽度和移位控制位数。

32 位数据最多需要移动 31 位，因此需要 5 位移位量：

$$shamt[4:0]$$

可以将 32 位桶形移位器设计为 5 级结构：

- 第 1 级控制是否移 1 位；
- 第 2 级控制是否移 2 位；
- 第 3 级控制是否移 4 位；
- 第 4 级控制是否移 8 位；
- 第 5 级控制是否移 16 位。

每一级都由一组 32 位宽的多路选择器实现，控制方式与 8 位桶形移位器一致，只是数据总线位宽从 8 位扩展到 32 位。通过逐级选择是否移位以及移位方向、移位类型，即可实现完整的 32 位桶形移位器。

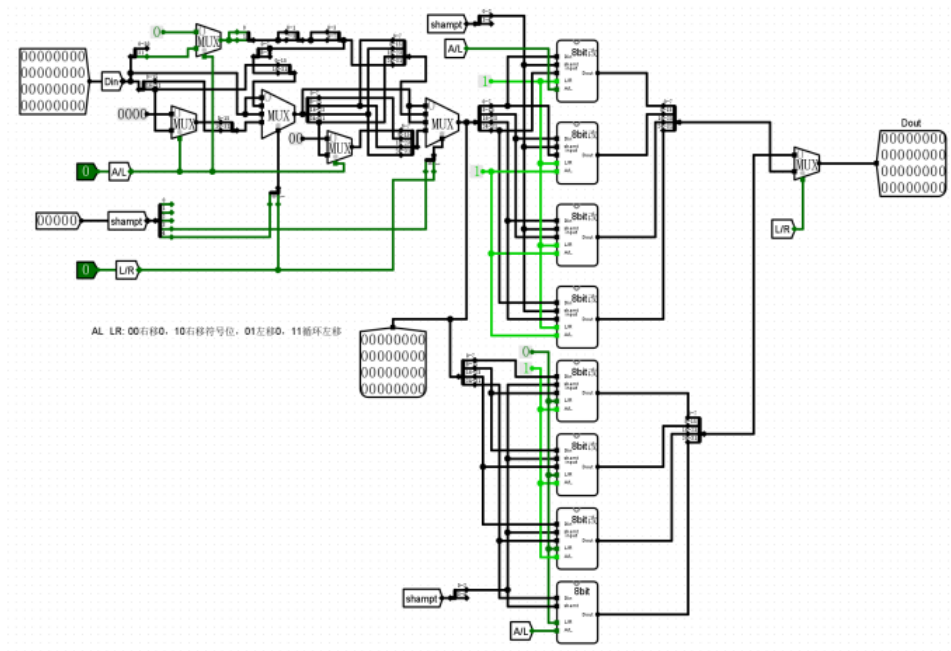


图 16: 思考题 4